

2026 SISBID High-Dimensional Hypothesis Testing Demo

Genevera I. Allen and Yufeng Liu

Load packages

```
required_pkgs <- c("sda", "ggplot2")
missing_pkgs <- required_pkgs[!vapply(required_pkgs, requireNamespace, logical(1), quietly = TRUE)]

if (length(missing_pkgs) > 0) {
  stop(
    "Please install the following packages before knitting this demo: ",
    paste(missing_pkgs, collapse = ", ")
  )
}

library(sda)
library(ggplot2)

theme_set(theme_minimal(base_size = 13))
```

We will consider many feature-wise tests of the form

$$H_0 : \text{feature is not associated with the response.}$$

The main goal of this demo is to show why unadjusted p-values can be misleading when many hypotheses are tested, and how procedures such as Bonferroni and Benjamini–Hochberg behave in simple examples.

Helper functions

```
feature_t_tests <- function(x, y) {
  # x should be features x samples.
  # y should contain two class labels, one per sample.
  y <- factor(y)
  stopifnot(ncol(x) == length(y), nlevels(y) == 2)

  apply(x, 1, function(z) t.test(z ~ y)$p.value)
}

bh_rejection_count <- function(p_values, alpha = 0.1) {
  p_ordered <- sort(p_values)
  m <- length(p_ordered)
  hits <- which(p_ordered <= alpha * seq_len(m) / m)
  if (length(hits) == 0) 0 else max(hits)
}

bonferroni_rejection_count <- function(p_values, alpha = 0.1) {
```

```

    sum(p_values <= alpha / length(p_values))
  }

plot_adjusted_vs_raw <- function(p_values, title) {
  plot_df <- data.frame(
    OriginalP = p_values,
    BHAdjustedP = p.adjust(p_values, method = "BH")
  )

  ggplot(plot_df, aes(x = OriginalP, y = BHAdjustedP)) +
    geom_point(alpha = 0.55) +
    geom_abline(intercept = 0, slope = 1, linetype = "dashed") +
    coord_equal(xlim = c(0, 1), ylim = c(0, 1)) +
    labs(
      title = title,
      x = "Original p-values",
      y = "BH-adjusted p-values"
    )
}

plot_ordered_p <- function(p_values, alpha = 0.1, n_show = 100, title = NULL) {
  p_ordered <- sort(p_values)
  m <- length(p_ordered)
  n_show <- min(n_show, m)

  plot_df <- data.frame(
    Index = seq_len(n_show),
    OrderedP = p_ordered[seq_len(n_show)],
    BHBoundary = alpha * seq_len(n_show) / m,
    BonferroniBoundary = alpha / m
  )

  ggplot(plot_df, aes(x = Index, y = OrderedP)) +
    geom_point(size = 1.8, alpha = 0.75) +
    geom_line(aes(y = BHBoundary), linewidth = 0.8) +
    geom_hline(yintercept = alpha / m, linetype = "dashed") +
    labs(
      title = title,
      subtitle = paste0(
        "BH line and Bonferroni threshold shown at alpha = ", alpha
      ),
      x = "Index of ordered p-value",
      y = "Ordered p-value"
    )
}

plot_bh_adjusted_by_index <- function(p_values, title) {
  plot_df <- data.frame(
    Index = seq_along(p_values),
    BHAdjustedP = p.adjust(p_values, method = "BH")
  )

  ggplot(plot_df, aes(x = Index, y = BHAdjustedP)) +

```

```

    geom_point(alpha = 0.55) +
    labs(
      title = title,
      x = "Feature index",
      y = "BH-adjusted p-value"
    )
}

```

Data set 1: simulated data, all null hypotheses true

In this first example, none of the features are truly associated with the binary response. Even so, when we run 1000 tests, we should expect some small p-values by chance.

```

n_features <- 1000
n_samples <- 50

x <- matrix(rnorm(n_features * n_samples), nrow = n_features, ncol = n_samples)
y <- sample(c(0, 1), n_samples, replace = TRUE)

p_values_null <- feature_t_tests(x, y)

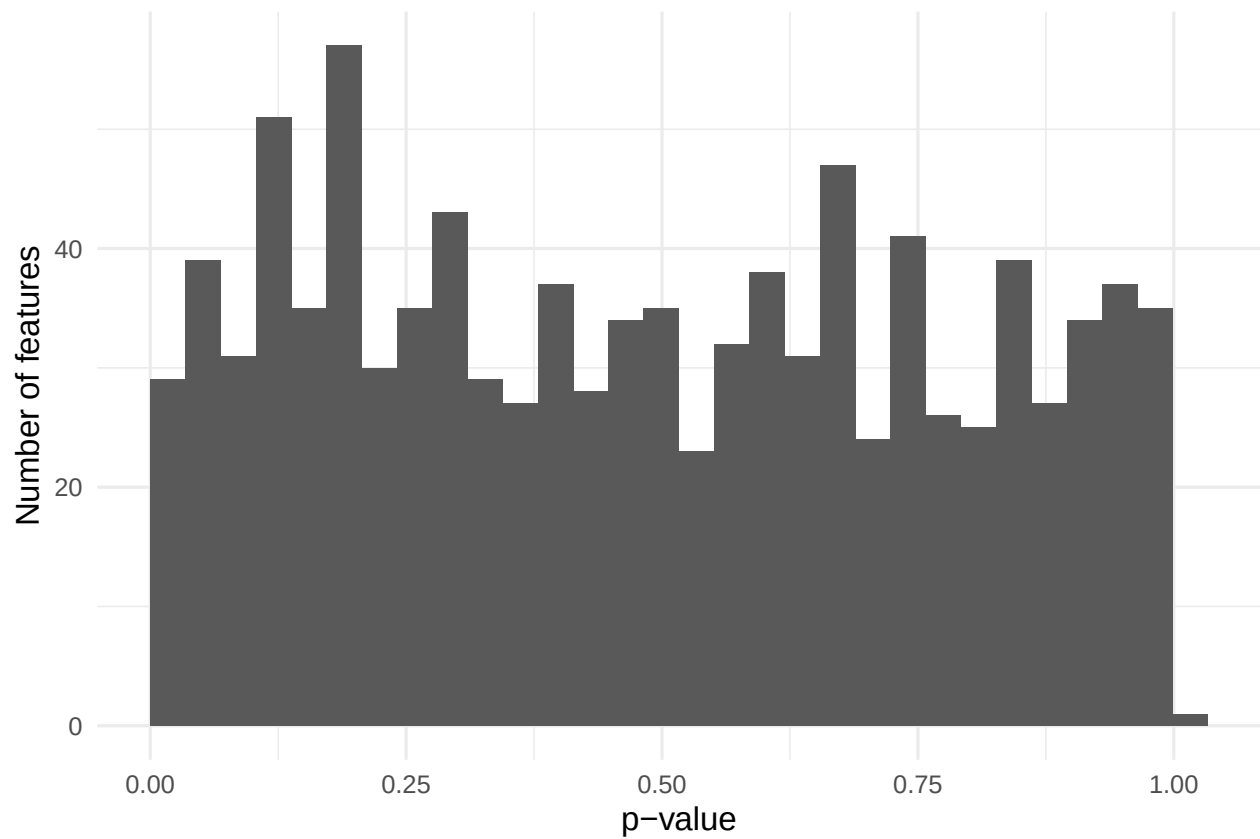
cat(
  "Proportion of unadjusted p-values below 0.05:",
  round(mean(p_values_null < 0.05), 3),
  "\n"
)

## Proportion of unadjusted p-values below 0.05: 0.044

ggplot(data.frame(PValue = p_values_null), aes(x = PValue)) +
  geom_histogram(boundary = 0, bins = 30) +
  labs(
    title = "All-null simulation: p-values are approximately uniform",
    x = "p-value",
    y = "Number of features"
  )

```

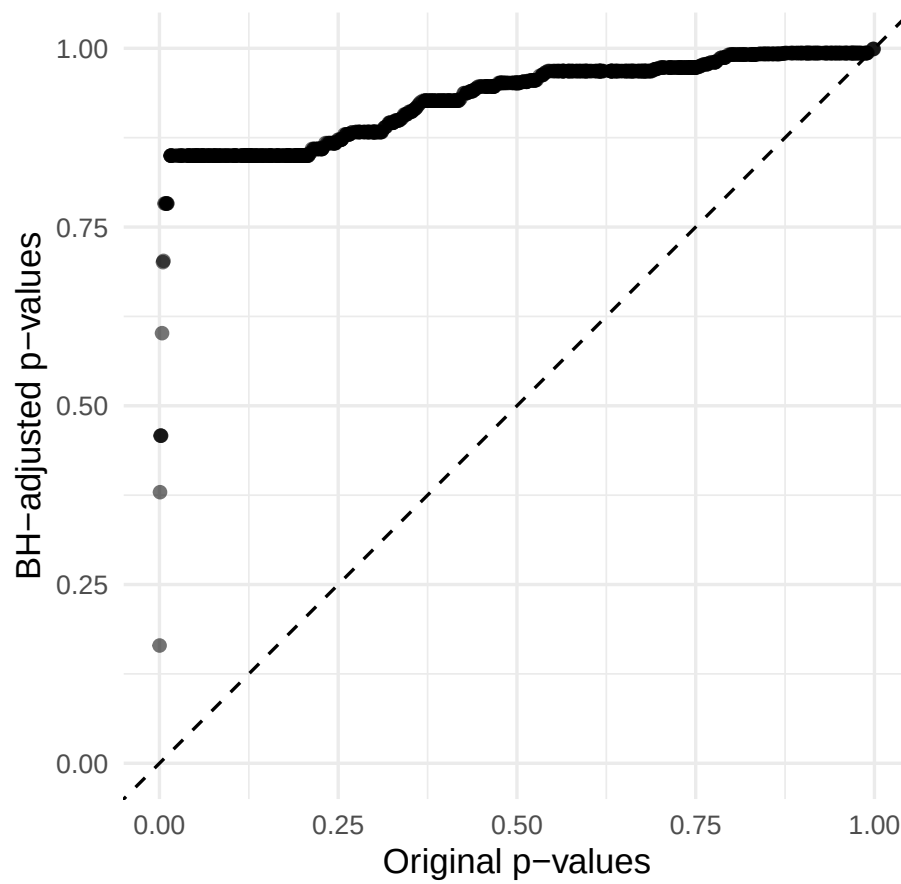
All-null simulation: p-values are approximately uniform



Benjamini–Hochberg adjustment

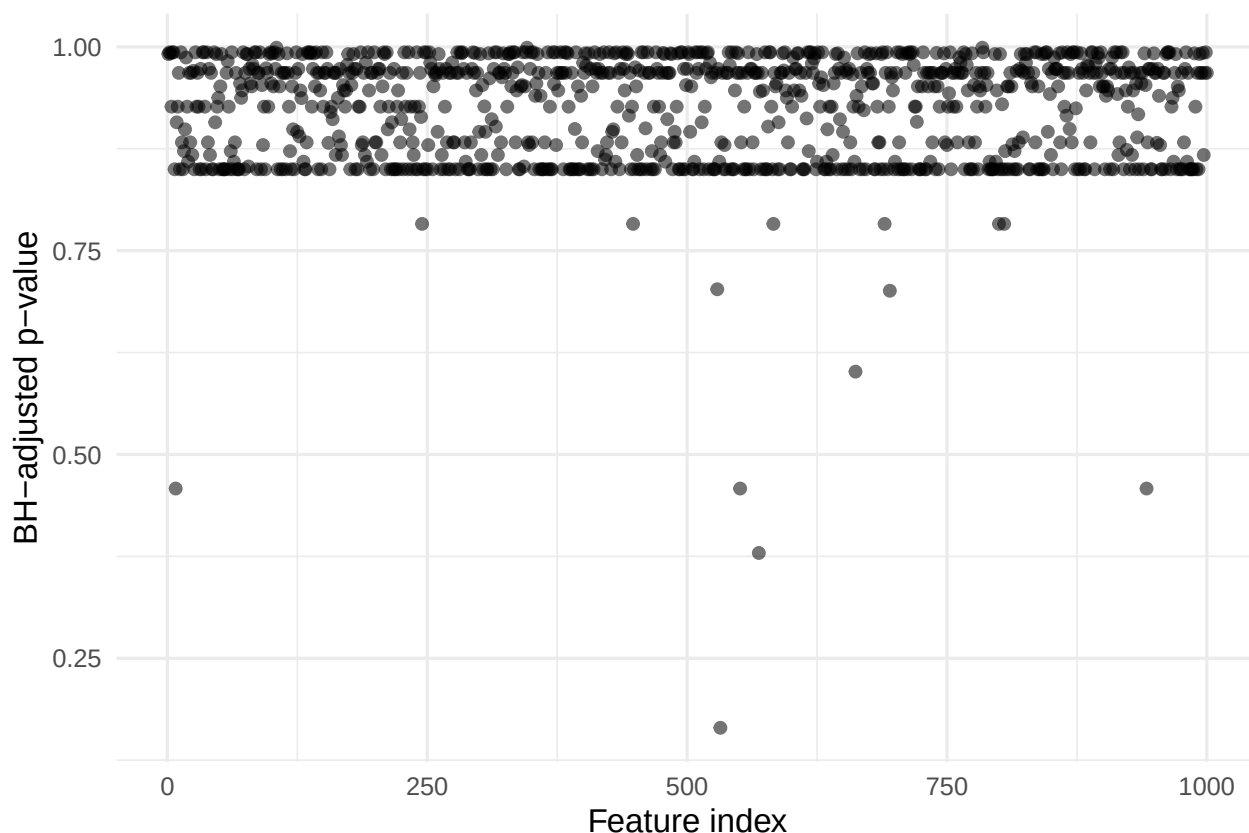
```
plot_adjusted_vs_raw(  
  p_values_null,  
  title = "All-null simulation: original versus BH-adjusted p-values"  
)
```

All-null simulation: original versus BH-adjusted p-val



```
plot_bh_adjusted_by_index(  
  p_values_null,  
  title = "All-null simulation: BH-adjusted p-values by feature index"  
)
```

All-null simulation: BH-adjusted p-values by feature index



Data set 2: simulated data, some alternatives true

In this second example, the first 100 features have a shifted mean in one group. Now not all null hypotheses are true, and we should see many more small p-values.

```
x <- matrix(rnorm(n_features * n_samples), nrow = n_features, ncol = n_samples)
y <- sample(c(0, 1), n_samples, replace = TRUE)

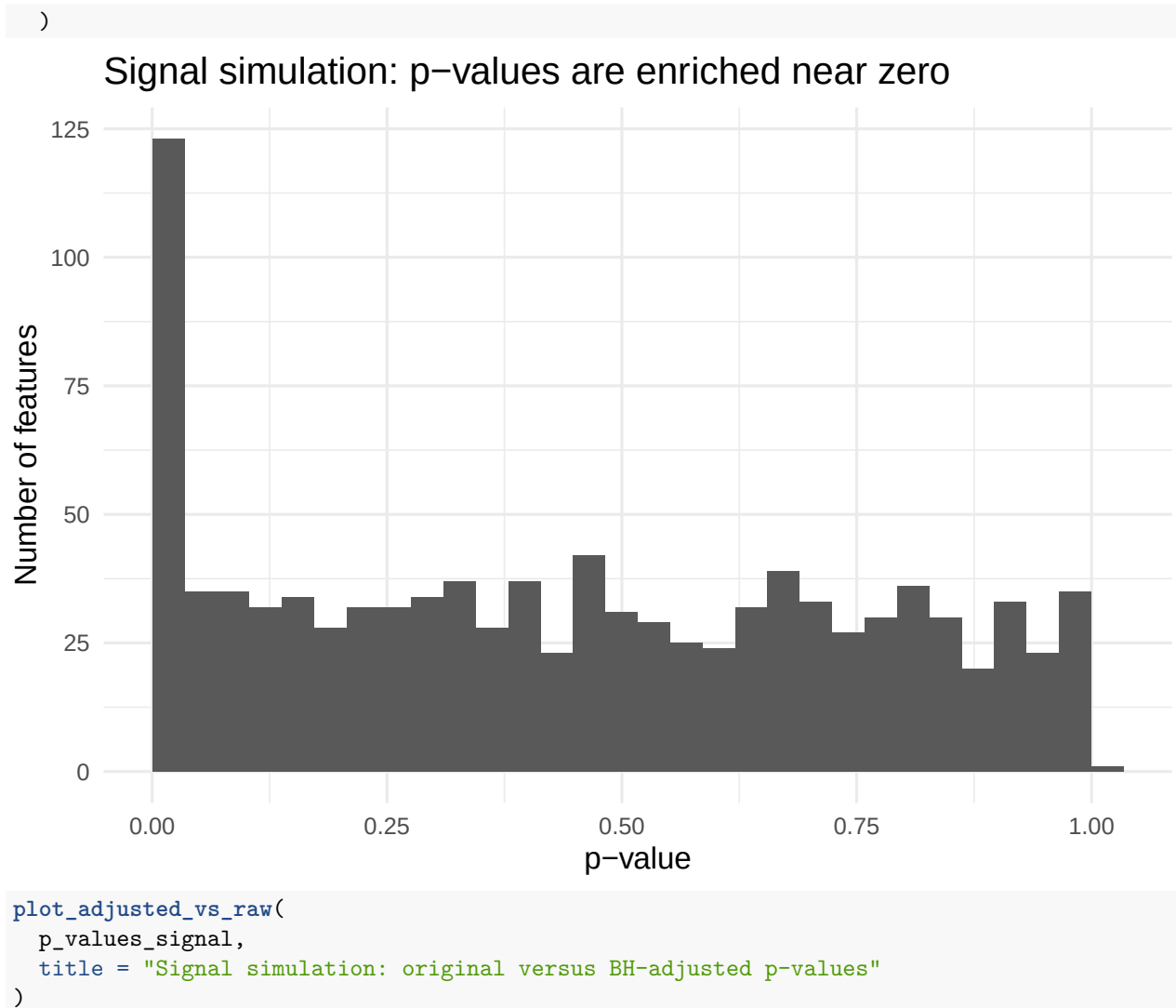
signal_features <- 1:100
x[signal_features, y == 0] <- x[signal_features, y == 0] + 1

p_values_signal <- feature_t_tests(x, y)

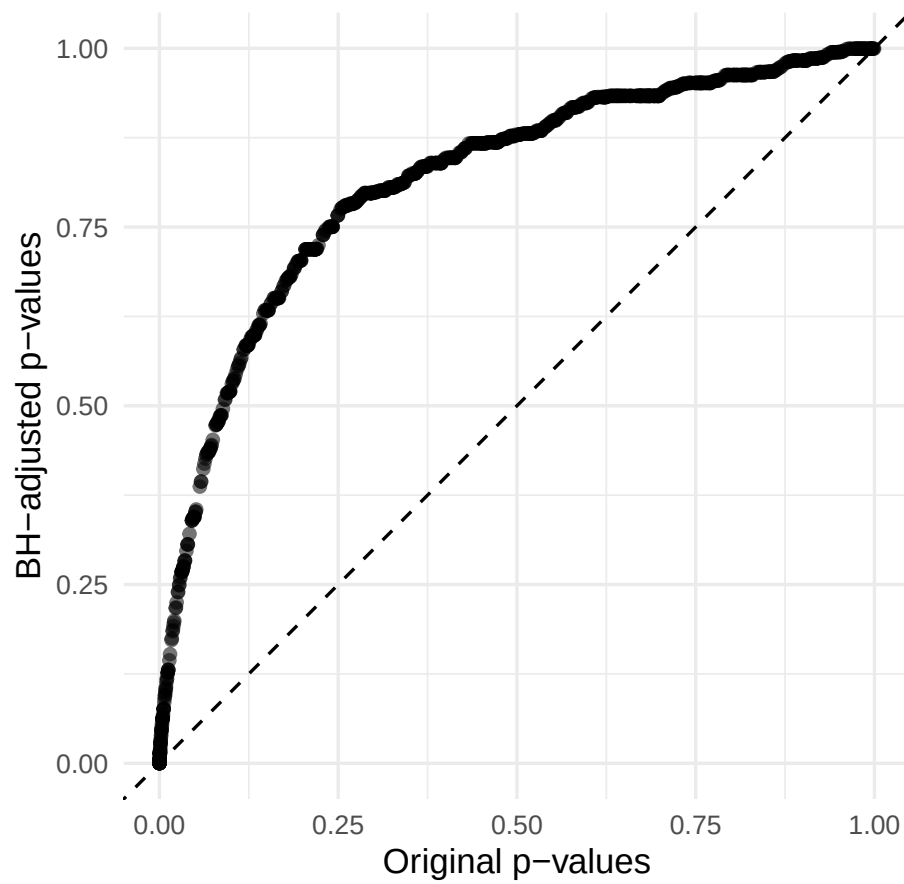
cat(
  "Proportion of unadjusted p-values below 0.05:",
  round(mean(p_values_signal < 0.05), 3),
  "\n"
)
```

```
## Proportion of unadjusted p-values below 0.05: 0.142
```

```
ggplot(data.frame(PValue = p_values_signal), aes(x = PValue)) +
  geom_histogram(boundary = 0, bins = 30) +
  labs(
    title = "Signal simulation: p-values are enriched near zero",
    x = "p-value",
    y = "Number of features"
  )
```

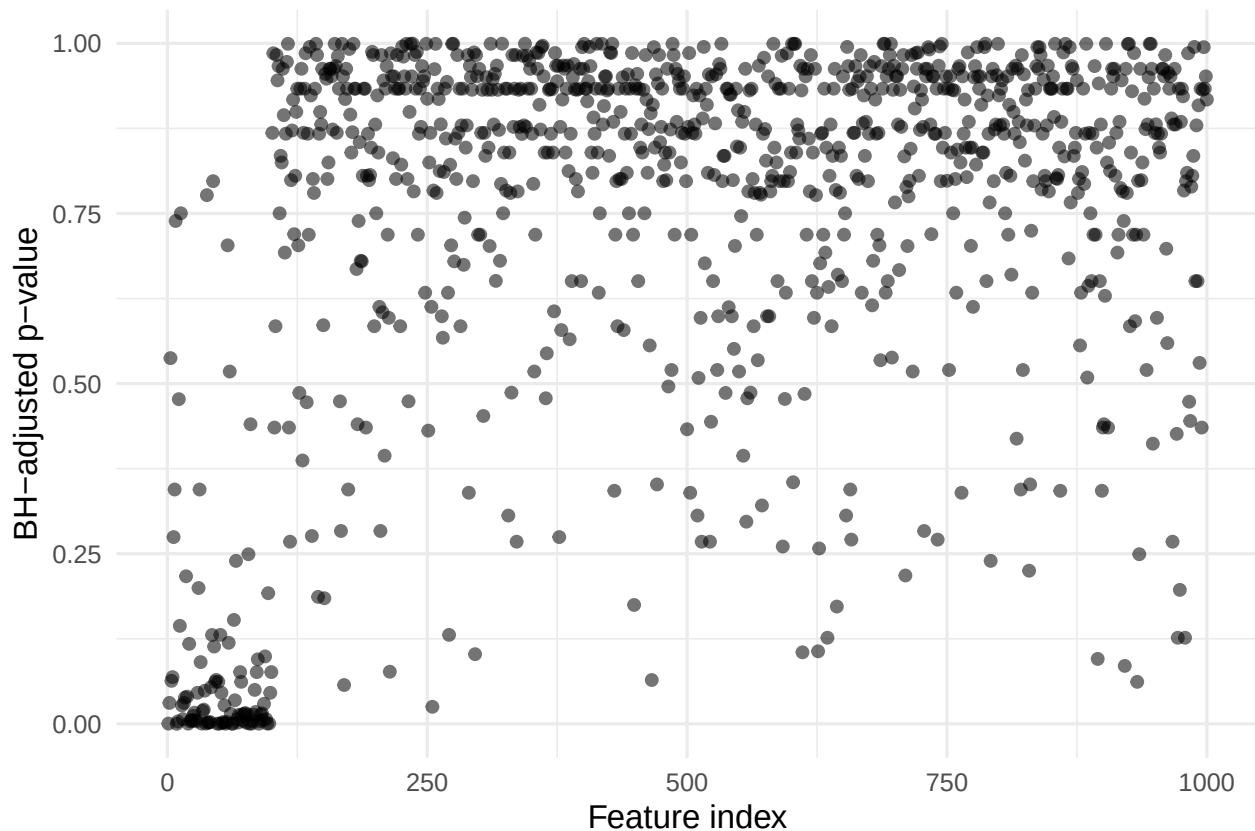


Signal simulation: original versus BH-adjusted p-values



```
plot_bh_adjusted_by_index(  
    p_values_signal,  
    title = "Signal simulation: BH-adjusted p-values by feature index"  
)
```


Signal simulation: BH-adjusted p-values by feature index



Directly visualizing the BH rule

The BH procedure rejects all hypotheses up to the largest ordered p-value below the line

$$p_{(i)} \leq \alpha \frac{i}{m}.$$

Bonferroni rejects p-values below the horizontal threshold α/m .

```
alpha <- 0.4

fdrs_bh_signal <- p.adjust(p_values_signal, method = "BH")

cat(
  "Number of tests with BH-adjusted p-value below", alpha, ":",
  sum(fdrs_bh_signal < alpha),
  "\n"
)

## Number of tests with BH-adjusted p-value below 0.4 : 148

cat(
  "Number of BH rejections computed directly:",
  bh_rejection_count(p_values_signal, alpha = alpha),
  "\n"
)

## Number of BH rejections computed directly: 148
```

```

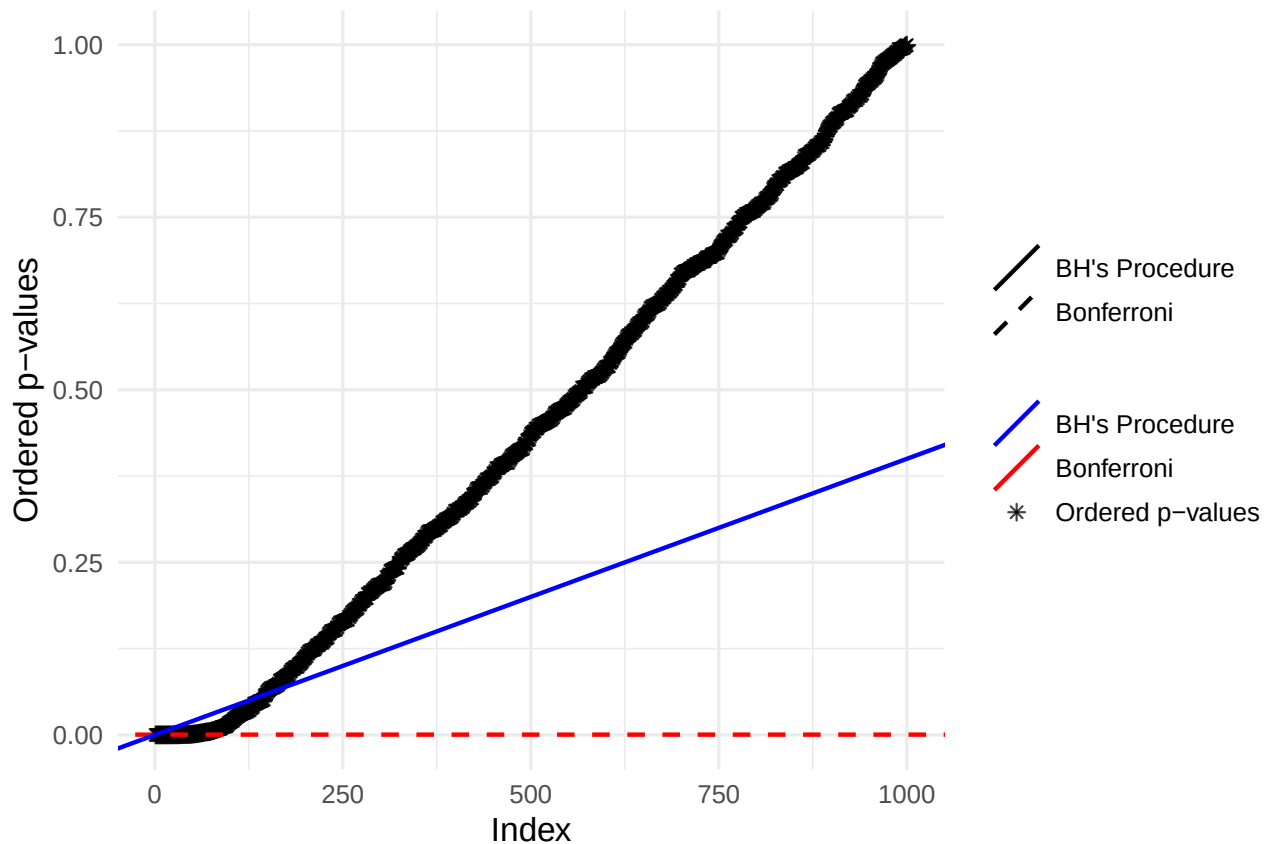
# This follows the original demo plot: order the raw p-values, then
# overlay the Bonferroni horizontal threshold and the BH step-up boundary.
BHData <- data.frame(
  OriginalP = p_values_signal,
  BH.P = fdrs_bh_signal
)
BHData <- BHData[order(BHData$OriginalP, decreasing = FALSE), ]
BHData$index <- seq_len(nrow(BHData))

boundary_df <- data.frame(
  Method = c("Bonferroni", "BH's Procedure"),
  intercept = c(alpha / nrow(BHData), 0),
  slope = c(0, alpha / nrow(BHData))
)

ggplot(BHData, aes(x = index, y = OriginalP)) +
  geom_point(aes(color = "Ordered p-values"), shape = 8, alpha = 0.75) +
  geom_abline(
    data = boundary_df,
    aes(intercept = intercept, slope = slope, color = Method, linetype = Method),
    linewidth = 0.8
  ) +
  scale_color_manual(
    name = NULL,
    values = c(
      "Bonferroni" = "red",
      "BH's Procedure" = "blue",
      "Ordered p-values" = "black"
    ),
    breaks = c("BH's Procedure", "Bonferroni", "Ordered p-values")
  ) +
  scale_linetype_manual(
    name = NULL,
    values = c("Bonferroni" = "dashed", "BH's Procedure" = "solid"),
    breaks = c("BH's Procedure", "Bonferroni")
  ) +
  labs(
    title = "Signal simulation: ordered p-vals with BH and Bonferroni",
    x = "Index",
    y = "Ordered p-values"
  ) +
  theme(plot.title = element_text(hjust = 0.5))

```

Signal simulation: ordered p-vals with BH and Bonferroni



Data set 3: real data, prostate cancer expression data

The prostate cancer data set from Singh et al. (2002) contains gene expression measurements for 6033 genes across 102 men. The data set is available from the R package `sda`.

In this example, we test each gene for differential expression between healthy and cancer samples.

```
data(singh2002)

x <- singh2002$x
y <- factor(singh2002$y)

n_healthy <- sum(y == "healthy")
n_cancer <- length(y) - n_healthy

cat("Number of healthy samples:", n_healthy, "\n")

## Number of healthy samples: 50
cat("Number of cancer samples:", n_cancer, "\n")

## Number of cancer samples: 52
cat("Number of genes:", ncol(x), "\n")

## Number of genes: 6033
```

```

# x is samples x genes for the prostate data.
# Use the labels directly rather than assuming the samples are ordered by class.
p_values_prostate <- apply(x, 2, function(z) t.test(z ~ y)$p.value)

if (is.null(colnames(x))) {
  gene_id <- paste0("Gene_", seq_len(ncol(x)))
} else {
  gene_id <- colnames(x)
}

prostate_results <- data.frame(
  Gene = gene_id,
  PValue = p_values_prostate,
  BHAdjustedP = p.adjust(p_values_prostate, method = "BH"),
  BonferroniAdjustedP = p.adjust(p_values_prostate, method = "bonferroni")
)

prostate_results <- prostate_results[order(prostate_results$PValue), ]

knitr::kable(
  head(prostate_results, 10),
  digits = 4,
  caption = "Top genes ranked by raw p-value"
)

```

Table 1: Top genes ranked by raw p-value

	Gene	PValue	BHAdjustedP	BonferroniAdjustedP
610	Gene_610	0	0.0011	0.0011
1720	Gene_1720	0	0.0048	0.0095
364	Gene_364	0	0.0151	0.0594
914	Gene_914	0	0.0151	0.0699
332	Gene_332	0	0.0151	0.0858
3940	Gene_3940	0	0.0151	0.0906
4546	Gene_4546	0	0.0183	0.1282
1068	Gene_1068	0	0.0188	0.1732
579	Gene_579	0	0.0188	0.1978
4331	Gene_4331	0	0.0188	0.2053

```

alpha <- 0.1
m_prostate <- length(p_values_prostate)
p1 <- sort(p_values_prostate)

cat(
  "Number of Bonferroni rejections at alpha =", alpha, ":",
  bonferroni_rejection_count(p_values_prostate, alpha = alpha),
  "\n"
)

## Number of Bonferroni rejections at alpha = 0.1 : 6

cat(
  "Number of BH rejections at FDR level", alpha, ":",

```

```

    bh_rejection_count(p_values_prostate, alpha = alpha),
    "\n"
)

## Number of BH rejections at FDR level 0.1 : 57

# This keeps the original base-R plot style and adds the legend explicitly.
plot(
  p1[1:100],
  pch = rep("*", 100),
  ylim = c(0, 0.003),
  xlab = "Index",
  ylab = "ordered p-values",
  main = "Prostate data: ordered p-values with BH and Bonferroni boundaries"
)

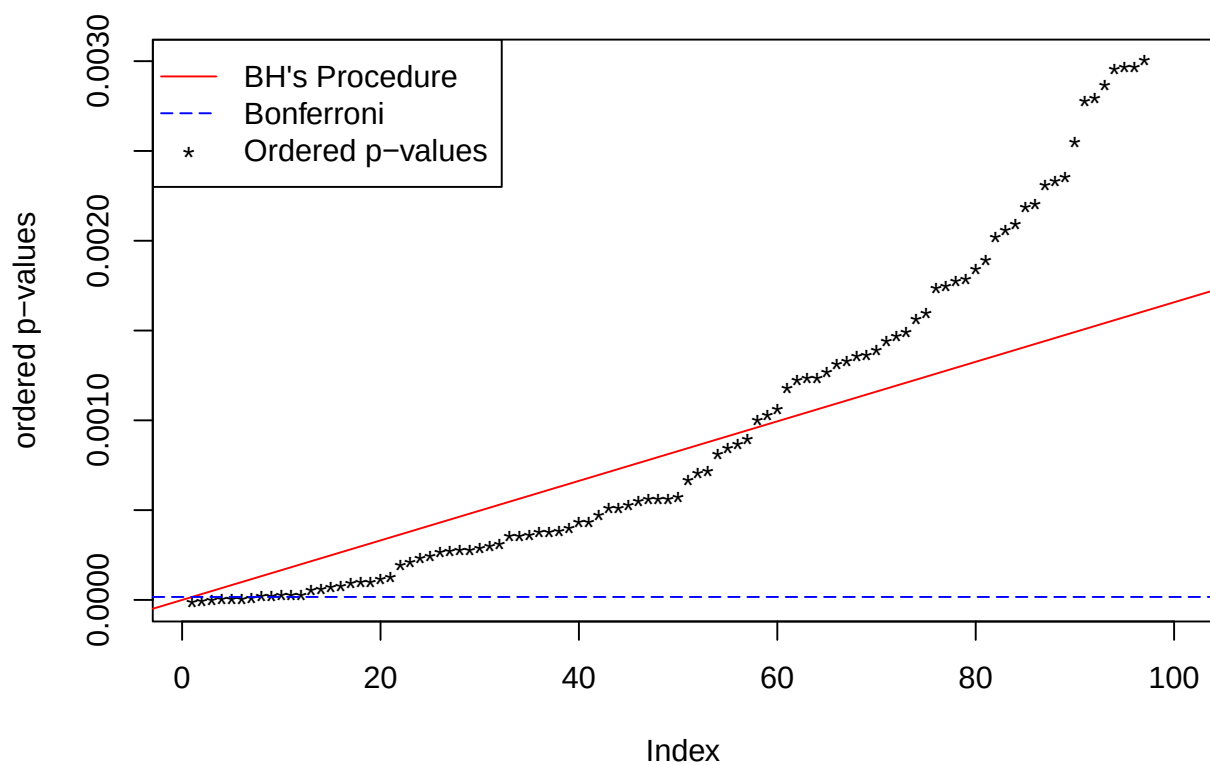
# Rejection boundary of Benjamini-Hochberg's procedure.
abline(a = 0, b = alpha / m_prostate, col = "red")

# Rejection boundary of Bonferroni at alpha.
abline(a = alpha / m_prostate, b = 0, col = "blue", lty = 5)

legend(
  "topleft",
  legend = c("BH's Procedure", "Bonferroni", "Ordered p-values"),
  lty = c(1, 5, NA),
  col = c("red", "blue", "black"),
  pch = c(NA, NA, "*")
)

```

Prostate data: ordered p-values with BH and Bonferroni boundaries



```
plot_adjusted_vs_raw(  
  p_values_prostate,  
  title = "Prostate data: original versus BH-adjusted p-values"  
)
```

Prostate data: original versus BH-adjusted p-values

